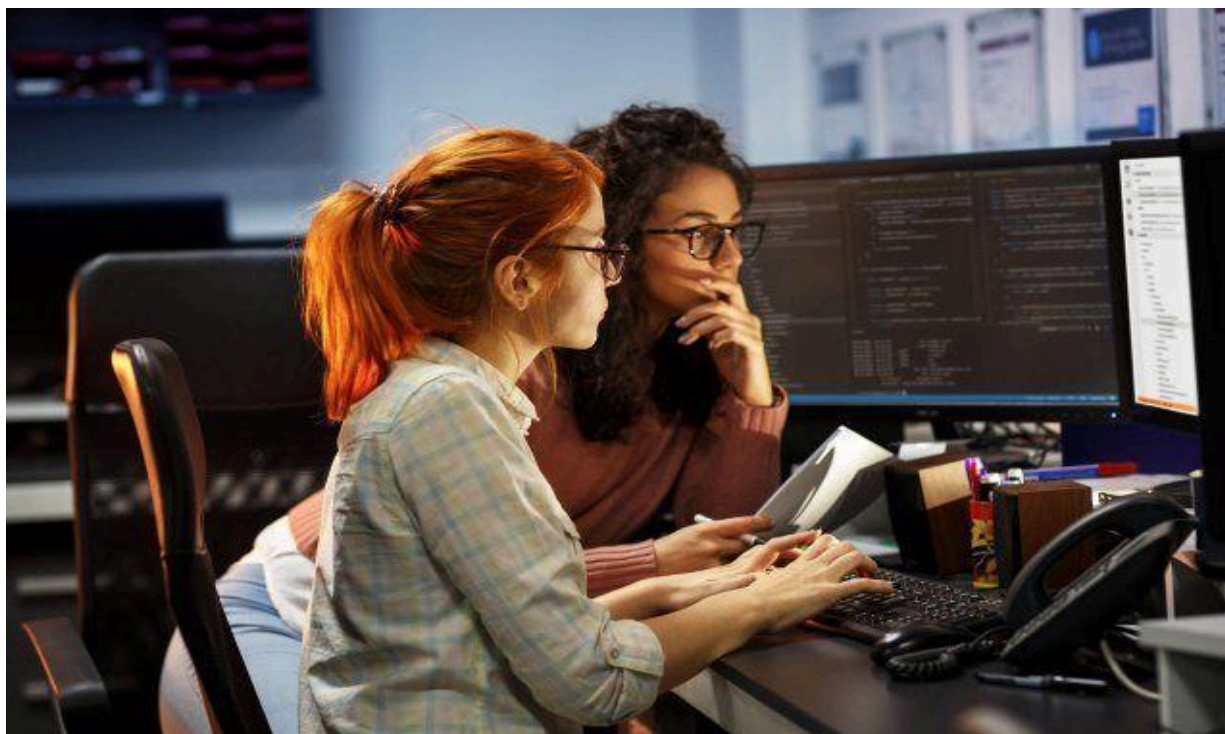




Universidad de El Salvador

Facultad Multidisciplinaria de Occidente – Ingeniería en Desarrollo de Software

Asignatura:
Programación Orientada a Objetos

Unidad #: 4

Tema: Persistencia, Archivos y Concurrency Moderna

SEMANA 12

Autor:

Introducción:

Bienvenidos al material de lectura de la Unidad #4 de la materia Programación Orientada a Objetos.

En este documento exploraremos los principios fundamentales de la concurrencia en Java, incluyendo el manejo de hilos (threads), su ciclo de vida, la sincronización, las prioridades, los grupos de hilos, y el uso de clases modernas como `ExecutorService`, `Callable`, `Future` y streams paralelos.

Este material ha sido diseñado para brindarte una base sólida en el desarrollo de aplicaciones multitarea y concurrentes, preparándote para enfrentar los desafíos de la programación moderna y orientada al rendimiento.

A medida que avances en el contenido, te animamos a experimentar activamente con los ejemplos, prácticas y ejercicios propuestos, ya que te ayudarán a comprender cómo optimizar la ejecución de tareas y aprovechar al máximo los recursos del sistema.

Semana 12

4.0 Concurrency básica

La concurrencia es la capacidad de un programa para ejecutar múltiples tareas de manera simultánea o intercalada. En el contexto de la ingeniería de software moderna, especialmente en aplicaciones empresariales y de servicios web, la concurrencia se ha convertido en un requisito fundamental.

En Java, la concurrencia se logra principalmente mediante el uso de hilos (threads), que son unidades independientes de ejecución dentro de un proceso. Un proceso puede contener múltiples hilos que comparten el mismo espacio de memoria, lo que permite comunicación eficiente pero requiere sincronización cuidadosa.

¿Por qué es importante la concurrencia?

- **Aprovechamiento de hardware multinúcleo:** Los procesadores modernos tienen múltiples núcleos. La concurrencia permite usar todos estos núcleos simultáneamente.
- **Mejor experiencia de usuario:** Las interfaces gráficas permanecen responsivas mientras se realizan operaciones pesadas en segundo plano.
- **Eficiencia en operaciones I/O:** Mientras un hilo espera datos de red o disco, otros pueden continuar trabajando.
- **Escalabilidad en servidores:** Los servidores pueden atender múltiples clientes simultáneamente, esencial para aplicaciones web y servicios.
- **Procesamiento de Big Data:** Permite dividir grandes conjuntos de datos para procesamiento paralelo.

Conceptos fundamentales

Proceso vs Hilo:

Un proceso es una instancia de un programa en ejecución con su propio espacio de memoria. Los hilos son unidades de ejecución dentro de un proceso que comparten memoria pero tienen su propia pila de ejecución.

Concurrencia vs Paralelismo:

La concurrencia es sobre estructura (múltiples tareas en progreso), el paralelismo es sobre ejecución (múltiples tareas ejecutándose simultáneamente en diferentes núcleos).

4.1. Aplicaciones Multitarea en Java.

Una **aplicación multitarea** es aquella capaz de **ejecutar varias tareas de manera concurrente** dentro del mismo programa.

En Java, esto se logra mediante el uso de **hilos (threads)**, que permiten que diferentes partes del código se ejecuten al mismo tiempo o de forma alternada, aprovechando los recursos del procesador de manera más eficiente.

Por ejemplo, un programa puede:

- Descargar un archivo mientras actualiza la interfaz gráfica.
- Procesar datos mientras escucha peticiones de red.
- Reproducir música mientras el usuario navega en el menú.

Multitarea en Java

Java soporta multitarea de dos maneras principales:

1. Multitarea basada en procesos:

- Cada proceso tiene su propio espacio de memoria y recursos.
- Es más pesado (consume más recursos del sistema).
- Ejemplo: ejecutar varios programas a la vez (Eclipse, Chrome, VSCode).

2. Multitarea basada en hilos (multihilo o multithreading):

- Varios hilos se ejecutan dentro del mismo proceso compartiendo memoria.

- Es más ligero y eficiente.
- Ejemplo: un programa que lee un archivo mientras responde al usuario.

Ventajas del multithreading

- **Mayor rendimiento:** aprovecha mejor los procesadores multinúcleo.
- **Mayor fluidez:** las tareas pesadas no bloquean el resto de la aplicación.
- **Reactividad:** mejora la respuesta ante eventos del usuario o de red.
- **Aislamiento:** los hilos pueden ejecutar diferentes funciones sin interferir directamente.

Cómo crear multitarea en Java

```
class MiHilo extends Thread {  
    public void run() {  
        System.out.println("Hilo en ejecución: " + Thread.currentThread().getName());  
    }  
}
```

```
public class Ejemplo1 {  
    public static void main(String[] args) {  
        MiHilo h1 = new MiHilo();  
        MiHilo h2 = new MiHilo();  
        h1.start();  
        h2.start();  
    }  
}
```

Cada vez que se invoca `start()`, se ejecuta el método `run()` en un hilo independiente.

implementar la interfaz `Runnable`

```
class MiTarea implements Runnable {  
    public void run() {  
        System.out.println("Ejecutando hilo: " +  
Thread.currentThread().getName());  
    }  
}  
  
public class Ejemplo2 {  
    public static void main(String[] args) {  
        Thread t1 = new Thread(new MiTarea());  
        Thread t2 = new Thread(new MiTarea());  
        t1.start();  
        t2.start();  
    }  
}
```

Esta forma es más flexible y se prefiere cuando la clase ya hereda de otra

```
import java.util.concurrent.ExecutorService;

import java.util.concurrent.Executors;


public class Ejemplo3 {

    public static void main(String[] args) {

        ExecutorService executor =
        Executors.newFixedThreadPool(2);


        executor.submit(() -> System.out.println("Tarea 1
ejecutada por " + Thread.currentThread().getName()));

        executor.submit(() -> System.out.println("Tarea 2
ejecutada por " + Thread.currentThread().getName()));


        executor.shutdown();

    }

}
```

El **ExecutorService** gestiona automáticamente los hilos y mejora la eficiencia.

4.1.1 Creación de Hilos

En Java existen dos formas principales de crear hilos. Ambas son válidas, pero tienen diferentes casos de uso y ventajas.

Método 1: Extendiendo la clase Thread

Esta es la forma más directa. Creamos una clase que hereda de Thread y sobrescribimos su método run(). El método run() contiene el código que se ejecutará cuando iniciemos el hilo.

Ejemplo básico:

```
class MiHilo extends Thread {  
    private String nombre;  
  
    public MiHilo(String nombre) {  
        this.nombre = nombre;  
    }  
  
    @Override  
    public void run() {  
        System.out.println(nombre + " iniciado");  
        try {  
            Thread.sleep(2000);  
        } catch (InterruptedException e) {  
            e.printStackTrace();  
        }  
        System.out.println(nombre + " finalizado");  
    }  
}
```

```
}
```

```
// Uso:
```

```
MiHilo hilo1 = new MiHilo("Hilo-A");
```

```
MiHilo hilo2 = new MiHilo("Hilo-B");
```

```
hilo1.start(); // Inicia el hilo
```

```
hilo2.start();
```

Método 2: Implementando la interfaz Runnable

Esta forma es más flexible porque Java no permite herencia múltiple. Al usar Runnable, nuestra clase puede extender de otra clase si es necesario. Además, promueve mejor separación entre la tarea a ejecutar y el mecanismo de ejecución.

Ejemplo con Runnable:

```
class TareaProcesamiento implements Runnable {
```

```
    private String datos;
```

```
    public TareaProcesamiento(String datos) {
```

```
        this.datos = datos;
```

```
    }
```

```
    @Override
```

```
    public void run() {
```

```
        System.out.println("Procesando: " + datos);
```

```
        // Lógica de procesamiento
```

```
    }
```



```
}
```

```
// Uso:
```

```
Thread hilo = new Thread(new TareaProcesamiento("Datos UES"));  
  
hilo.start();
```

Creación de hilos con Expresiones Lambda.

A partir de Java 8, el lenguaje introdujo las expresiones lambda, una forma más concisa y moderna de implementar interfaces funcionales.

Dado que la interfaz Runnable es funcional (solo tiene un método: run()), podemos crear hilos de manera más simple y legible.

```
// Lambda simple
```

```
Thread hilo1 = new Thread(() -> {  
  
    System.out.println("Tarea ejecutándose");  
  
});  
  
hilo1.start();
```

Importante: Siempre llama a start(), NO a run(). Llamar a run() directamente ejecuta el código en el hilo actual (no crea un nuevo hilo), mientras que start() crea un nuevo hilo de ejecución

Ejemplo Práctico UES: Sistema de Registro de Estudiantes

Imaginemos un sistema de inscripción de la UES donde múltiples estudiantes intentan registrarse simultáneamente en diferentes materias:

```
class RegistroEstudiante implements Runnable {  
  
    private String carnet;  
  
    private String materia;
```

```
public RegistroEstudiante(String carnet, String materia) {  
  
    this.carnet = carnet;  
  
    this.materia = materia;  
  
}  
  
@Override  
  
public void run() {  
  
    System.out.println("Estudiante " + carnet +  
        " inscribiendo " + materia);  
  
    try {  
        Thread.sleep(1500); // Simula proceso  
        System.out.println("Inscripción exitosa: " +  
            carnet + " en " + materia);  
    } catch (InterruptedException e) {  
        System.out.println("Error en inscripción");  
    }  
}  
}
```

// Uso en sistema de inscripciones:

```
Thread est1 = new Thread(new RegistroEstudiante(  
    "AA12001", "Programación Concurrente"));  
  
Thread est2 = new Thread(new RegistroEstudiante(  
    "AA12002", "Programación Concurrente"));
```

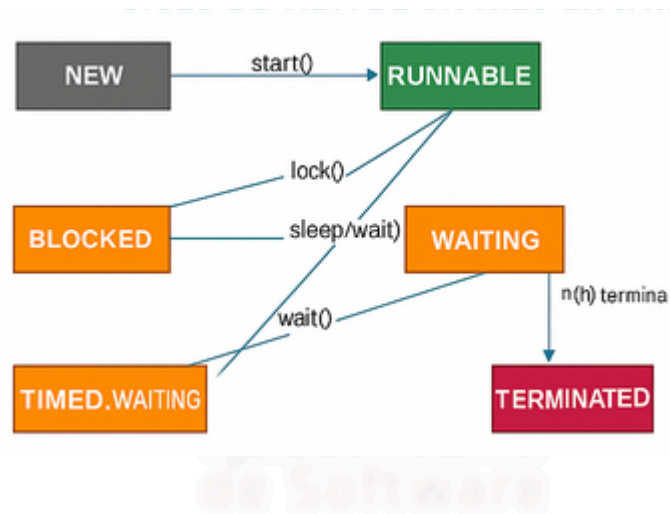
"BB13002", "Programación Concurrente"));

```
est1.start();
```

```
est2.start();
```

4.1.2 Ciclo de Vida de los Hilos

Un hilo en Java pasa por varios estados durante su existencia. Entender estos estados es fundamental para depurar problemas de concurrencia y optimizar el rendimiento de aplicaciones multihilo.



Estados del ciclo de vida:

1. NEW (Nuevo)

El hilo ha sido creado pero aún no se ha iniciado. El método `start()` no ha sido invocado. En este estado, el hilo existe como un objeto Java pero no como un hilo del sistema operativo.

```
Thread hilo = new Thread(() -> {});
```

```
// Estado: NEW
```

```
System.out.println(hilo.getState()); // NEW
```

2. RUNNABLE (Ejecutable)

El hilo está listo para ejecutarse o ejecutándose activamente. Después de llamar a `start()`, el hilo entra en este estado. El planificador del sistema operativo decide cuándo ejecutarlo realmente.

```
hilo.start();
```

```
// Estado: RUNNABLE (listo o ejecutándose)
```

3. BLOCKED (Bloqueado)

El hilo está esperando para adquirir un monitor (lock) para entrar en una sección sincronizada o método sincronizado. Esto ocurre cuando otro hilo ya posee el lock.

```
synchronized(objeto) {
```

```
    // Si otro hilo tiene el lock,
```

```
    // este hilo estará BLOCKED
```

```
}
```

4. WAITING (Esperando)

El hilo está esperando indefinidamente a que otro hilo realice una acción específica. Se entra a este estado mediante `wait()`, `join()` sin timeout, o `LockSupport.park()`.

```
synchronized(objeto) {
```

```
    objeto.wait(); // Estado: WAITING
```

```
}
```

5. TIMED_WAITING (Esperando con tiempo)

El hilo está esperando por un tiempo específico. Se entra mediante `sleep()`, `wait(timeout)`, `join(timeout)`, o `parkNanos()`.

```
Thread.sleep(1000); // TIMED_WAITING por 1 seg
```

6. TERMINATED (Terminado)

El hilo ha completado su ejecución. El método `run()` ha finalizado, ya sea normalmente o por una excepción no capturada. Un hilo terminado no puede ser reiniciado.

Diagrama de transiciones:

NEW → `start()` → RUNNABLE

RUNNABLE → sincronización bloqueada → BLOCKED

RUNNABLE → `wait()/join()` → WAITING

RUNNABLE → `sleep()/wait(timeout)` → TIMED_WAITING

BLOCKED/WAITING/TIMED_WAITING → condición satisfecha → RUNNABLE

RUNNABLE → `run()` termina → TERMINATED

4.1.2.1 Métodos para Control de Threads

Java proporciona varios métodos para controlar la ejecución de hilos. Estos métodos permiten coordinar la ejecución, pausar hilos, y esperar por la finalización de otros hilos.

`sleep(milisegundos)`

Pausa el hilo actual por el tiempo especificado en milisegundos. El hilo pasa al estado `TIMED_WAITING` y libera el procesador, pero NO libera los locks que posee.

```
try {
```

```
    System.out.println("Iniciando pausa");
```

```
Thread.sleep(3000); // Pausa 3 segundos

System.out.println("Pausa completada");

} catch (InterruptedException e) {

    System.out.println("Hilo interrumpido");

    Thread.currentThread().interrupt();

}
```

join()

Hace que el hilo actual espere hasta que el hilo especificado termine su ejecución. Es útil cuando necesitamos que una tarea se complete antes de continuar.

```
Thread trabajador = new Thread(() -> {

    System.out.println("Trabajador procesando...");

    try {

        Thread.sleep(2000);

    } catch (InterruptedException e) {}

    System.out.println("Trabajo completado");

});
```

```
trabajador.start();
```

```
try {

    trabajador.join(); // Espera que termine

    System.out.println("Trabajador ha terminado");

} catch (InterruptedException e) {

    e.printStackTrace();

}
```

join(timeout)

Variante de join() que espera un máximo de tiempo especificado:

```
trabajador.join(5000); // Espera máximo 5 segundos
```

```
if (trabajador.isAlive()) {  
    System.out.println("Aún no termina");  
}
```

interrupt()

Interrumpe un hilo que está en estado WAITING o TIMED_WAITING. El hilo debe manejar la InterruptedException apropiadamente. Es la forma recomendada de cancelar hilos.

```
Thread hilo = new Thread(() -> {  
    try {  
        while (!Thread.interrupted()) {  
            System.out.println("Trabajando...");  
            Thread.sleep(1000);  
        }  
    } catch (InterruptedException e) {  
        System.out.println("Hilo interrumpido");  
    }  
});
```

```
hilo.start();
```

```
Thread.sleep(3000);
```

```
hilo.interrupt(); // Solicita interrupción
```

yield()

Sugiere al planificador que el hilo actual está dispuesto a ceder su tiempo de CPU a otros hilos. Es solo una sugerencia y no garantiza que otro hilo se ejecute. Útil en algoritmos cooperativos.

```
for (int i = 0; i < 100; i++) {  
  
    System.out.println("Procesando: " + i);  
  
    Thread.yield(); // Cede el procesador  
  
}
```

Ejemplo Práctico UES: Sistema de Consulta de Notas

Simulemos un sistema donde múltiples estudiantes consultan sus notas simultáneamente, usando los métodos de control de hilos:

```
class ConsultaNotas implements Runnable {  
  
    private String carnet;  
  
    public ConsultaNotas(String carnet) {  
        this.carnet = carnet;  
    }  
  
    @Override  
    public void run() {  
        try {  
            System.out.println(carnet +  
                ": Conectando al servidor...");  
            Thread.sleep(1000);  
  
            System.out.println(carnet +
```



```
        ": Autenticando...");  
  
        Thread.sleep(500);  
  
        System.out.println(carnet +  
            ": Recuperando notas...");  
  
        Thread.sleep(1500);  
  
        System.out.println(carnet +  
            ": Notas recuperadas exitosamente");  
    } catch (InterruptedException e) {  
        System.out.println(carnet +  
            ": Consulta cancelada");  
    }  
}  
  
}  
  
// Uso del sistema:  
  
Thread est1 = new Thread(new ConsultaNotas("AA15001"));  
Thread est2 = new Thread(new ConsultaNotas("BB16002"));  
Thread est3 = new Thread(new ConsultaNotas("CC17003"));  
  
est1.start();  
est2.start();  
est3.start();
```

```
// Esperar a que todos terminen  
  
est1.join();  
  
est2.join();  
  
est3.join();  
  
System.out.println("Todas las consultas completadas");
```

4.1.3. Sincronización

Cuando múltiples hilos acceden a recursos compartidos simultáneamente, pueden ocurrir condiciones de carrera (race conditions) donde el resultado depende del orden impredecible de ejecución de los hilos. La sincronización resuelve este problema mediante el control de acceso a secciones críticas del código.

El problema: Condiciones de carrera

Consideremos un contador compartido sin sincronización:

```
1  class ContadorInseguro {  
2      private int cuenta = 0;  
3  
4      public void incrementar() {  
5          cuenta++; // NO es thread-safe  
6          // Esta operación tiene 3 pasos:  
7          // 1. Leer valor de cuenta  
8          // 2. Incrementar el valor  
9          // 3. Escribir el nuevo valor  
10     }  
11  
12     public int getCuenta() {  
13         return cuenta;  
14     }  
15 }
```

Si dos hilos ejecutan `incrementar()` simultáneamente, pueden leer el mismo valor, incrementarlo, y escribir el resultado, perdiendo un incremento. Por ejemplo, ambos leen 5, incrementan a 6, y escriben 6, cuando debería ser 7.

La solución: `synchronized`

La palabra clave `synchronized` asegura que solo un hilo pueda ejecutar el código sincronizado a la vez. Cada objeto Java tiene un monitor (lock) implícito que `synchronized` utiliza.

Método 1: Métodos sincronizados

```
class ContadorSeguro { no usages
    private int cuenta = 0; 2 usages

    public synchronized void incrementar() { no usages
        cuenta++; // Ahora es thread-safe
    }

    public synchronized int getCuenta() { no usages
        return cuenta;
    }
}
```

Método 2: Bloques sincronizados

Para mayor control y mejor rendimiento, sincroniza solo la sección crítica:

```
class ContadorOptimizado { no usages
    private int cuenta = 0; 1 usage
    private Object lock = new Object(); 1 usage

    public void incrementar() { no usages
        // Código no crítico aquí
        System.out.println("Preparando incremento");

        synchronized(lock) {
            // Solo esto está sincronizado
            cuenta++;
        }

        // Más código no crítico
    }
}
```

Ejemplo UES: Sistema de Reserva de Laboratorios

Simulemos el sistema de reserva de laboratorios de computación de la FIA-UES, donde múltiples estudiantes intentan reservar espacios simultáneamente:

```
class LaboratorioFIA { no usages
    private int espaciosDisponibles = 30; 3 usages
    private String nombre; 2 usages

    public LaboratorioFIA(String nombre) { no usages
        this.nombre = nombre;
    }

    public synchronized boolean reservar(String carnet) { no usages
        if (espaciosDisponibles > 0) {
            System.out.println(carnet +
                " reservando en " + nombre);

            espaciosDisponibles--;

            System.out.println("Reserva exitosa. Espacios: " +
                espaciosDisponibles);
            return true;
        } else {
            System.out.println(carnet +
                ": Lab lleno");
            return false;
        }
    }
}
```

Comunicación entre hilos: wait(), notify(), notifyAll()

Estos métodos permiten la comunicación y coordinación entre hilos. Deben usarse dentro de bloques synchronized:

- **wait():** Libera el lock y pone el hilo en estado WAITING hasta ser notificado
- **notify():** Despierta UN hilo que está esperando en el mismo objeto
- **notifyAll():** Despierta TODOS los hilos que están esperando

Ejemplo: Productor-Consumidor UES

Simulemos un buffer compartido donde un hilo productor genera datos (ej: trabajos de impresión) y un consumidor los procesa:

